

Engineering Production-Grade Retrieval-Augmented Generation: Architectures for Scalable, Multi-Tenant AI Services

The evolution of Retrieval-Augmented Generation (RAG) from experimental prototypes to production-grade enterprise services demands a rigorous transition from simple vector search to complex, stateful, and secure architectures. At the core of this transition lies the need for robust multi-tenant isolation, high-fidelity linguistic processing for agglutinative languages such as Turkish, and sophisticated document engineering strategies like parent-child chunking. To achieve industrial reliability, developers are increasingly converging on a stack comprising FastAPI for asynchronous orchestration, PostgreSQL for relational integrity, and Qdrant for high-performance vector operations. This report analyzes the architectural requirements, implementation strategies, and open-source foundations necessary to build such a system, focusing on repositories with significant community traction and production-worthy patterns.¹

Architectural Foundations of Multi-Tenant Isolation

Multi-tenancy in a RAG context involves the logical and physical separation of data between different organizational entities, ensuring that even in a shared infrastructure, data leakage is architecturally impossible. This is particularly challenging in vector databases where indices are often optimized for global similarity rather than partitioned access. The current industry gold standard for this isolation involves a two-layer approach: relational Row-Level Security (RLS) in PostgreSQL and payload-based filtering in Qdrant.⁴

PostgreSQL Row-Level Security and Relational Integrity

For the relational components of a RAG service—including user accounts, organization metadata, document ownership records, and chat histories—PostgreSQL Row-Level Security (RLS) provides a kernel-level guarantee of isolation. Unlike application-level filtering, where a developer might inadvertently omit a `WHERE tenant_id = ?` clause in a complex join, RLS policies are enforced by the database engine itself.⁵ This provides a defense-in-depth security posture, essential for highly regulated industries.

The implementation of RLS typically follows a pattern where the application sets a session-local variable, such as `app.tenant_id`, immediately after authenticating a request. The database then applies a policy to all subsequent queries within that session. This mechanism ensures that even if an attacker manages to execute arbitrary SQL, they remain confined within the boundaries of their own tenant's data. For a FastAPI service, this context is often managed via a

dependency that wraps every database transaction, ensuring the tenant context is set before any business logic is executed.²

Vector Store Isolation and Payload Filtering in Qdrant

In the vector domain, Qdrant facilitates multi-tenancy through the use of payload filters. Each vector point is enriched with a `tenant_id` and often a secondary `scope_id` for granular access control within an organization. While earlier versions of vector databases required separate collections per tenant—an approach that does not scale to thousands of users—modern tiered multi-tenancy in Qdrant (v1.16+) allows for a single collection with highly optimized internal partitioning based on a specific payload field.⁴

The security boundary in Qdrant is currently transitioning from application-enforced filtering to server-side enforcement via JSON Web Tokens (JWT). By embedding the `tenant_id` as a required claim in the JWT used to access the Qdrant API, the engine can internally merge a mandatory filter into every search, update, or delete request. This prevents "uncontrolled updates" where a user might attempt to overwrite or access data belonging to a different tenant by modifying the filter parameters in the query payload.⁴

Repository	GitHub URL	Area Covered	Specific Files/Folders	What NOT to Copy
Onetime Secret	github.com/onetimesecret/onetimesecret	PostgreSQL RLS Implementation	db/migrate/ (conditional DDL), apps/base_application.rb (session context)	The specific Ruby/Sequel syntax is less relevant; focus on the SQL DDL for CREATE POLICY and SET LOCAL patterns. ⁵
Onyx (Danswer)	github.com/danswer-ai/danswer	Multi-tenant ACL & Permission Mirroring	backend/onyx/db/ (ACL logic), backend/onyx/access/ (Permission checks)	The enterprise features are partially gated; avoid copying the specific licensing checks and focus on the

				document permissioning logic. ⁷
Qdrant Alloy	github.com/qdrant/qdrant-all oy	Multi-Tenant Native Vector Search	src/index.py (partition-based logic), configs/	This is a relatively new and lightweight repo; avoid using it as a full backend replacement, but rather as a reference for partition-based logic. ⁶

Turkish Linguistic Processing and Hybrid Search Optimization

Turkish presents significant challenges for traditional retrieval systems due to its agglutinative nature. A single word can have thousands of variations through the addition of suffixes, making standard keyword matching ineffective. To address this, a production RAG service must implement a hybrid search strategy that fuses dense semantic embeddings with sparse lexical tokens processed through a language-specific stemmer.⁶

The Mechanics of Turkish Hybrid Retrieval

The core of effective Turkish search lies in the use of the Snowball stemmer during the indexing and query phases. The stemming process reduces words to their semantic roots, allowing a query for "kitapçı" (bookseller) to match a document containing "kitaplarım" (my books) if the root "kitap" is indexed. However, lexical matching alone often fails to capture the intent of a query. Dense embeddings, such as those provided by multilingual-e5-large or BGE-M3, complement this by mapping phrases into a high-dimensional semantic space where "hukuk" (law) and "avukat" (lawyer) are proximal regardless of their lexical similarity.⁶

Qdrant enables this through a multi-vector collection architecture where both dense and sparse vectors are stored in parallel. The retrieval process uses a scoring fusion technique, often Reciprocal Rank Fusion (RRF) or a weighted sum of the scores. The BM25 algorithm, which serves as the foundation for the sparse component, is mathematically defined to weight the importance of terms based on their frequency in the document ($f(q_i, D)$) and their rarity

in the overall corpus (Inverse Document Frequency or IDF).¹²

$$\text{score}(D, Q) = \sum_{q_i \in Q} \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot (1 - b + b \cdot \frac{|D|}{\text{avgdl}})}$$

In a RAG environment, where document chunks are often of a fixed size, the traditional BM25 components for document length normalization ($|D|/\text{avgdl}$) become less critical. This has led to the emergence of newer sparse models like BM42, which leverage the attention weights of transformer models to identify important tokens more intelligently than simple frequency-based methods.¹²

Reranking for Precision

The final stage of the retrieval pipeline involves a cross-encoder reranker. While bi-encoders (dense retrieval) are efficient for searching millions of documents, they do not consider the fine-grained interaction between the query and the document. A reranker, such as the Cohere Reranker, takes the top 50-100 results from the hybrid search and performs a more computationally expensive analysis to determine the final top-k context chunks for the LLM. This step is critical for eliminating "distractor" chunks that may be semantically similar but factually irrelevant to the specific user question.³

Repository	GitHub URL	Area Covered	Specific Files/Folders	What NOT to Copy
Qdrant FastEmbed	github.com/qdrant/fastembed	Sparse/Dense Vector Generation	fastembed/sparse/bm25.py, fastembed/text/	Avoid the default English stop-word lists; ensure you integrate the Turkish-specific Snowball configurations. 6
RAG-Fundamentals	github.com/Abeshith/RAG-Fundamentals	Hybrid Search Fusion & Reranking	hybrid_search_fusion/, reranking_techniques/	This is a tutorial repo; do not copy the simplistic

				synchronous indexing loops for production use. ¹⁴
--	--	--	--	--

Document Engineering: Parent-Child Chunking and Async Pipelines

In production RAG, the quality of generation is inextricably linked to the structural integrity of the ingested data. Standard fixed-size chunking often fragments semantic units, leading to a loss of context. Parent-child chunking resolves this by decoupling the retrieval unit from the generation unit.¹⁵

Hierarchical Data Modeling with Explicit Foreign Keys

Parent-child chunking involves splitting a document into large "parent" chunks (e.g., 2000-4000 tokens) that preserve broad context, and then further subdividing those into small "child" chunks (e.g., 200-500 tokens) for dense embedding. During retrieval, the system searches the child chunks for the highest similarity but returns the text of the associated parent chunk to the LLM. This provides the generator with a comprehensive view of the surrounding information, reducing hallucinations and improving the coherence of the response.¹⁵

From a database perspective, this requires an explicit foreign key relationship in PostgreSQL. A chunks table stores the child vectors and metadata, while a parent_documents or document_sections table stores the larger text blocks. When a search result is returned from Qdrant, its metadata includes the parent_id, which is then used to perform a high-speed relational lookup to retrieve the generative context. This architecture ensures that the system maintains a "ground truth" relational structure alongside the probabilistic vector search.²

Asynchronous Ingestion with ARQ and Redis

Processing thousands of documents—especially those requiring complex parsing of tables and images—cannot be done synchronously within the FastAPI request-response cycle. A production ingestion pipeline utilizes ARQ or Celery with Redis as a task broker to manage these long-running jobs. The architecture typically follows a "fan-out" pattern:

1. **Request Entry:** FastAPI receives a document, uploads it to object storage (MinIO/S3), and creates a job record in PostgreSQL.
2. **Job Enqueueing:** A background task is enqueued in ARQ with the document reference and tenant context.
3. **Worker Processing:** The ARQ worker retrieves the document, performs parsing (using tools like Marker or OmniParse), executes the hierarchical chunking strategy, generates embeddings, and performs batch upserts into Qdrant and PostgreSQL.¹⁸

This asynchronous approach allows for sophisticated error handling and job tracking. If an embedding model's API is rate-limited, the worker can use exponential backoff to retry the operation without affecting the user's ability to continue using the application. Metrics such as job latency, success rates, and queue depth are tracked via Prometheus and visualized in Grafana to ensure the pipeline remains healthy under load.²

Repository	GitHub URL	Area Covered	Specific Files/Folders	What NOT to Copy
Agentic RAG for Dummies	github.com/GiovanniPasq/agentic-rag-for-dummies	Parent-Child Chunking Logic	project/db/vector_db_manager.py, project/db/parent_store_manager.py	Avoid the local JSON parent store; replace it with the PostgreSQL FK logic discussed in the narrative. ¹⁵
FastAPI RAG API	github.com/danny-avila/rag_api	Async Indexing & File ID Filtering	main.py (FastAPI routes), utils/ (Batching logic)	This repo has lower star counts but highly specific ID-based filtering logic; ignore the simplistic Docker configs. ¹⁸
RAGFlow	github.com/infiniflow/ragflow	Deep Parsing & Task Workers	api/db/models.py, task_executor.py	The multi-tenant visibility has reported limitations; focus on the high-fidelity document parsing and task scheduling

				logic. ¹⁶
--	--	--	--	----------------------

Audio Intelligence and Speaker Diarization Pipelines

The integration of audio data into a RAG service adds a temporal and identity-based dimension to the retrieval problem. The objective is to enable users to query spoken content with the same precision as text, including the ability to ask "What did the CEO say about Q4 results during the meeting?" This requires a pipeline that combines OpenAI’s Whisper for transcription with Pyannote for speaker diarization.²³

The Transcription and Diarization Workflow

Audio processing begins with Voice Activity Detection (VAD) to isolate speech from background noise and silences. Advanced pipelines then use a diarization model to extract speaker embeddings, which are clustered to identify unique speakers. One critical challenge in this process is timestamp accuracy; standard Whisper outputs can drift by several seconds in long recordings, which is unacceptable for precise diarization.²⁴

To solve this, a "forced alignment" step is introduced. Models like WhisperX use phoneme-level alignment to snap the transcribed text to exact timestamps in the audio waveform. This ensures that speaker turns are accurately associated with the correct words. The resulting data structure is a list of segments, each containing the text, speaker_id, start_time, and end_time. These segments are then chunked and embedded into Qdrant, allowing the RAG system to retrieve not just the text, but the specific moment and person associated with it.²⁴

Indexing these audio results requires a specialized schema in the vector store. Metadata should include the source audio URL and a timestamp offset. This enables the frontend to provide a "jump to audio" feature in the citations, where clicking a retrieved chunk plays the exact segment of the recording where the information was spoken.¹¹

Repository	GitHub URL	Area Covered	Specific Files/Folders	What NOT to Copy
WhisperX	github.com/m-bain/whisperx	Forced Alignment & Diarization	whisperx/alignment.py, whisperx/diarize.py	Avoid using the default "stable" PyPI release for bleeding-edge GPU support; follow the developer

				installation guide. ²⁴
Whisper Diarization	github.com/MahmoudAshraf97/whisper-diarization	Parallel NeMo/Whisper Execution	diarize.py, diarize_parallel.py	The parallel implementation is experimental; ensure you have the required 10GB+ VRAM before copying the parallel logic. ²⁵
pyannote-audio	github.com/pyannote/pyannote-audio	State-of-the-art Diarization	pyannote/audio/pipelines/	Do not attempt to re-train the models without massive datasets; use the pretrained pipelines from Hugging Face. ²³

Agentic Orchestration with LangGraph and Self-RAG

Modern RAG services are evolving from static retrieval chains to dynamic, agentic workflows. Instead of a single-shot query to the vector database, an agent can reason about the user's question, determine if more information is needed, and verify the quality of the retrieved context before generating a response. LangGraph is the premier framework for building these stateful, cyclic graphs.³

The Stateful Reasoning Loop

In a LangGraph-based "Self-RAG" implementation, the process is modeled as a state machine. The "State" object persists through the conversation, containing the current query, retrieved documents, and any intermediate reasoning steps. The graph typically includes several specialized nodes:

- **Query Rewriter:** If the initial retrieval yields poor results, an LLM rewrites the query to improve recall, often by resolving coreferences or expanding acronyms.²⁹
- **Hypothetical Document Embedding (HyDE):** The agent generates a "fake" ideal answer

to the user's question and then uses the embedding of that fake answer to find real documents in Qdrant. This bridges the semantic gap between a question and the declarative statements typically found in knowledge bases.²⁹

- **Relevance Grader:** A node that analyzes each retrieved document and assigns a binary or scalar score. Irrelevant documents are discarded, preventing the generator from being distracted by "noise".³
- **Hallucination Checker:** After generation, the agent verifies the answer against the retrieved documents. If the model has introduced information not found in the context, the node triggers a retry or a search expansion.²⁸

This cyclic nature allows for "Human-in-the-Loop" interactions. In high-stakes environments, the agent can pause its execution and wait for an administrator to approve the retrieved documents before proceeding to the generation phase. This level of control is impossible with linear LangChain pipelines.³

Repository	GitHub URL	Area Covered	Specific Files/Folders	What NOT to Copy
FastAPI LangGraph Template	github.com/wassim249/fastapi-langgraph-agent-production-ready-template	Production-Ready Agentic RAG	app/core/langgraph/graph.py, app/services/llm_service.py	The rate-limiting logic is robust but may need tuning for your specific API tier to avoid 429 errors. ²
Scalable RAG Pipeline	github.com/FareedKhan-dev/scalable-rag-pipeline	Agentic Reasoning & State Machines	src/agent/planner.py, src/agent/rewriter.py	The Terraform/AWS infrastructure is highly specific; focus on the Python agent logic for your FastAPI stack. ²⁹

Entity-Level Persistent Memory and Evaluation

The final frontier of production RAG is the implementation of long-term cognitive memory. While session history tracks the immediate conversation, entity-level memory (as seen in

Mem0) allows the agent to build a persistent profile of the user and their domain across weeks or months of interaction.³²

Semantic, Episodic, and Procedural Memory

Mem0 provides a three-layer memory architecture:

- 1. **Semantic Memory:** Extracts and stores facts and user preferences (e.g., "The user is a senior software engineer working on Turkish NLP").
- 2. **Episodic Memory:** Stores time-stamped task trajectories and past experiences to enable case-based reasoning.
- 3. **Procedural Memory:** Encodes patterns of tool use and execution logic.³²

The extraction process is governed by the A.U.D.N. cycle: **Add** new facts, **Update** existing knowledge, **Delete** contradictions, or **Noop** if the information is already known. This process is managed by an LLM-guided tool selection mechanism, ensuring that the memory layer remains consistent and relevant without manual intervention.³²

Observability with Langfuse and RAGAS

To maintain a production-grade service, engineers must quantify performance using the "RAG Triad": Context Relevance, Groundedness, and Answer Relevance. Langfuse captures the full execution traces of the LangGraph agent, while RAGAS uses these traces to perform model-based evaluation. This reference-free evaluation allows for continuous monitoring of production traffic, identifying instances where the retriever failed or the generator hallucinated without requiring a manually curated ground-truth dataset.³

Repository	GitHub URL	Area Covered	Specific Files/Folders	What NOT to Copy
Mem0	github.com/mem0ai/mem0	Entity-Level Long-Term Memory	mem0/memory/, mem0/embeddings/	Avoid the ephemeral "in-memory" instances for production; always configure the persistent vector store backend. ³³
RAGAS	github.com/vib	Evaluation	ragas/metrics/	Do not run

	rantlabsai/ragas	Metrics & Pipelines	ragas/llms/	evaluations on every single production request as it adds significant latency; use the batch sampling method. ³⁵
--	------------------	---------------------	-------------	---

Conclusion: Strategic Integration for Industrial Reliability

The construction of a production-grade RAG service requires a harmonious integration of security, linguistics, and orchestration. By leveraging PostgreSQL RLS and Qdrant payload filters, developers can ensure strict multi-tenant isolation. The adoption of Turkish-specific Snowball stemming and hybrid search architectures addresses the complexities of agglutinative languages. Furthermore, the use of LangGraph for stateful agentic flows and Mem0 for persistent entity-level memory transforms the RAG system from a passive search tool into an active, personalized cognitive assistant. Finally, a robust observability stack using Langfuse and RAGAS ensures that as the system scales, its performance and factual integrity are rigorously monitored and continuously improved.⁵

Alıntılanan çalışmalar

1. shanojpillai/qdrant-rag-pro: Building a Production-Ready RAG System with Qdrant Vector Database - GitHub, erişim tarihi Mart 13, 2026, <https://github.com/shanojpillai/qdrant-rag-pro>
2. wassim249/fastapi-langgraph-agent-production-ready ... - GitHub, erişim tarihi Mart 13, 2026, <https://github.com/wassim249/fastapi-langgraph-agent-production-ready-template>
3. Yigtwxx/Awesome-RAG-Production: A curated list of battle-tested tools, frameworks, and best practices for building scalable, production-grade Retrieval-Augmented Generation (RAG) systems. - GitHub, erişim tarihi Mart 13, 2026, <https://github.com/Yigtwxx/Awesome-RAG-Production>
4. Feature Request: Automatic Tenant-ID Injection via JWT Claims or Tenant-Specific API Keys · Issue #8015 - GitHub, erişim tarihi Mart 13, 2026, <https://github.com/qdrant/qdrant/issues/8015>
5. Implement PostgreSQL Row-Level Security (RLS) for Multi-Tenant Isolation #1876 - GitHub, erişim tarihi Mart 13, 2026, <https://github.com/onetimesecret/onetimesecret/issues/1876>

6. Qdrant Alloy - GitHub, erişim tarihi Mart 13, 2026, <https://github.com/qdrant/qdrant-alloy>
7. onyx-dot-app/onyx: Open Source AI Platform - AI Chat with ... - GitHub, erişim tarihi Mart 13, 2026, <https://github.com/danswer-ai/danswer>
8. Glean Alternatives: Complete 2026 Comparison [Interactive Matrix], erişim tarihi Mart 13, 2026, <https://www.thunai.ai/blog/glean-alternatives-and-competitors>
9. Getting started with semantic and hybrid search - OpenSearch Documentation, erişim tarihi Mart 13, 2026, <https://docs.opensearch.org/3.0/tutorials/vector-search/neural-search-tutorial/>
10. Text processing — list of Rust libraries/crates // Lib.rs, erişim tarihi Mart 13, 2026, <https://lib.rs/text-processing>
11. Multilingual RAG on a Podcast - Haystack - deepset AI, erişim tarihi Mart 13, 2026, https://haystack.deepset.ai/cookbook/multilingual_rag_podcast
12. BM42: New Baseline for Hybrid Search - Qdrant, erişim tarihi Mart 13, 2026, <https://qdrant.tech/articles/bm42/>
13. teppeisasaki/qdrant-bm25 - GitHub, erişim tarihi Mart 13, 2026, <https://github.com/teppeisasaki/qdrant-bm25>
14. ragas-evaluation · GitHub Topics, erişim tarihi Mart 13, 2026, <https://github.com/topics/ragas-evaluation?o=desc&s=stars>
15. agentic-rag-for-dummies/project/README.md at main - GitHub, erişim tarihi Mart 13, 2026, <https://github.com/GiovanniPasq/agentic-rag-for-dummies/blob/main/project/README.md>
16. Releases · infiniflow/ragflow - GitHub, erişim tarihi Mart 13, 2026, <https://github.com/infiniflow/ragflow/releases>
17. Danielskry/Awesome-RAG: Awesome list of Retrieval-Augmented Generation (RAG) applications in Generative AI. - GitHub, erişim tarihi Mart 13, 2026, <https://github.com/Danielskry/Awesome-RAG>
18. GitHub - danny-avila/rag_api: ID-based RAG FastAPI: Integration with Langchain and PostgreSQL/pgvector, erişim tarihi Mart 13, 2026, https://github.com/danny-avila/rag_api
19. cosimoiaia/FastAPI-RAG - GitHub, erişim tarihi Mart 13, 2026, <https://github.com/cosimoiaia/FastAPI-RAG>
20. XclusivVv/librechat-dashboard: Complete setup guide and ... - GitHub, erişim tarihi Mart 13, 2026, <https://github.com/XclusivVv/librechat-dashboard>
21. The Permission Issue of Knowledge Bases Created in Ragflow #10609 - GitHub, erişim tarihi Mart 13, 2026, <https://github.com/infiniflow/ragflow/issues/10609>
22. infiniflow/ragflow: RAGFlow is a leading open-source ... - GitHub, erişim tarihi Mart 13, 2026, <https://github.com/infiniflow/ragflow>
23. pyannote speaker diarization toolkit - GitHub, erişim tarihi Mart 13, 2026, <https://github.com/pyannote/pyannote-audio>
24. WhisperX: Automatic Speech Recognition with Word-level Timestamps (& Diarization) - GitHub, erişim tarihi Mart 13, 2026, <https://github.com/m-bain/whisperx>
25. MahmoudAshraf97/whisper-diarization: Automatic Speech Recognition with

- Speaker Diarization based on OpenAI Whisper - GitHub, erişim tarihi Mart 13, 2026, <https://github.com/MahmoudAshraf97/whisper-diarization>
26. Speker Diarization Using Whisper Transcription and Nvidia NeMo #898 - GitHub, erişim tarihi Mart 13, 2026, <https://github.com/openai/whisper/discussions/898>
27. An example LiveKit Voice Pipeline Agent with Qdrant VectorDB for RAG - GitHub, erişim tarihi Mart 13, 2026, <https://github.com/itsitgroup/livekit-qdrant-RAG-agent>
28. bootcamp/bootcamp/RAG/advanced_rag/langgraph-graphrag-agent-local.ipynb at master - GitHub, erişim tarihi Mart 13, 2026, https://github.com/milvus-io/bootcamp/blob/master/bootcamp/RAG/advanced_rag/langgraph-graphrag-agent-local.ipynb
29. FareedKhan-dev/scalable-rag-pipeline: A scalable RAG platform combining LangGraph agents, hybrid retrieval (Vector+Graph), and Ray orchestration on Kubernetes. - GitHub, erişim tarihi Mart 13, 2026, <https://github.com/FareedKhan-dev/scalable-rag-pipeline>
30. FastGPT vs RAGFlow: Which RAG Stack Wins for 2025 Deployments? - Sider.AI, erişim tarihi Mart 13, 2026, <https://sider.ai/blog/ai-tools/fastgpt-vs-ragflow-which-rag-stack-wins-for-2025-deployments>
31. langchain-ai/langgraph: Build resilient language agents as graphs. - GitHub, erişim tarihi Mart 13, 2026, <https://github.com/langchain-ai/langgraph>
32. A memory architecture for agentic system - GitHub Gist, erişim tarihi Mart 13, 2026, <https://gist.github.com/spikelab/7551c6368e23caa06a4056350f6b2db3>
33. Mem0 Tutorial: Persistent Memory Layer for AI Applications - DataCamp, erişim tarihi Mart 13, 2026, <https://www.datacamp.com/tutorial/mem0-tutorial>
34. GitHub All-Stars #2: Mem0 - Creating memory for stateless AI minds - VirtusLab, erişim tarihi Mart 13, 2026, <https://virtuslab.com/blog/ai/git-hub-all-stars-2/>
35. vibrantlabsai/ragas: Supercharge Your LLM Application Evaluations - GitHub, erişim tarihi Mart 13, 2026, <https://github.com/vibrantlabsai/ragas>
36. Evaluation of RAG pipelines with Ragas - Langfuse, erişim tarihi Mart 13, 2026, https://langfuse.com/guides/cookbook/evaluation_of_rag_with_ragas